LangChain and LangGraph Agentic AI Course Guide

An eight-week, hands-on path from model calls to reliable multi-agent applications

Prepared for a Python developer building RAG, digital-forensics, and surveillance applications.

Course length: 8 weeks | Suggested effort: 6–8 hours each week | Format: 5 study sessions per week

What you will be able to build

- LangChain applications that connect a model to prompts, structured outputs, document retrieval, and tools.
- A tool-using agent that decides when to call a function and uses the result to answer.
- LangGraph workflows with state, branches, loops, memory, and human approval pauses.
- Different patterns including RAG agents, routers, sequential workflows, planner and executor designs, handoffs, and supervisor with subagents.
- A final forensic investigation assistant that cites evidence and requires human review before consequential actions.

Recommended order: learn the pieces first, build one single-agent application, then use LangGraph when you need explicit control over the steps. LangChain's current agent interface is create_agent; its agents are built on LangGraph concepts. Avoid beginning with a large multi-agent system.

How to use this guide

Complete the sessions in order. Each session has a learning target, a task to implement, and a deliverable to save in your course repository. Spend about 60–90 minutes per session. If a task takes longer, finish it before moving on; the later weeks reuse your earlier code.

Create one Git repository with folders such as week01_basics, week02_chains, week03_rag, week04_tools, week05_graphs, week06_patterns, week07_quality, and capstone. Keep API keys in a .env file excluded from Git. Begin with a low-cost hosted model or a local model that supports tool calling; provider setup differs, but the core exercises do not.

Course map

Week	Focus	Weekly result
1	LLM and Python foundations	A tested model call and a small structured response
2	LangChain building blocks	Prompt-to-model pipeline and document indexing
3	RAG and retrieval quality	Grounded document Q&A with citations and test questions
4	Tools and single-agent patterns	A tool-using agent with safe, observable tool calls
5	LangGraph fundamentals	A custom graph with branches, loops, checkpointing, and pause/resume
6	Agent patterns	Router, workflow, planner/executor, handoff, and supervisor/subagents
7	Quality, safety, and serving	Evaluated, traced, guarded application behind a small API
8	Capstone	Forensic assistant with evidence sources, event summaries, and human review

Before Week 1

You do not need advanced machine learning to take this course. You should be comfortable writing Python functions, importing packages, reading JSON, calling a REST API, using virtual environments, and handling exceptions. Review these skills first if needed.

- Install Python 3.11 or newer, Git, and a code editor. Use a virtual environment for every project.
- Choose one model provider and confirm that its model supports the features needed for the exercises, especially structured output and tool calling.
- Create a .env file for credentials and add .env to .gitignore. Never commit API keys, private case material, or surveillance footage.
- Install packages as required by the official quickstarts. Keep a requirements.txt or pyproject.toml and record package versions after each working milestone.

API names and provider packages change over time. Follow the current official documentation for installation and model names. This guide teaches concepts and project tasks rather than pinning you to a provider.

Week 1 — Model calls and reliable Python foundations

Goal: understand the model request/response cycle before adding framework abstractions.

Session 1 — Prepare the workspace

Learn: Virtual environments, environment variables, API keys, Git, and project structure.

Task: Create the repository and environment. Add a .env.example containing placeholder names only, a .gitignore, README, and requirements file.

Deliverable: A clean repository that runs a minimal Python script.

Session 2 — Call a chat model

Learn: System and user messages, model parameters, token limits, and provider errors.

Task: Write a function that accepts a question, calls your chosen chat model, and returns the answer. Add timeout and error handling.

Deliverable: A model-call script with one successful example and one handled failure.

Session 3 — Prompt design

Learn: Instructions, context, examples, constraints, and prompt testing.

Task: Write three prompts for the same task: vague, clear, and example-based. Run each against five inputs and record where the output changes.

Deliverable: A small prompt experiment with observations.

Session 4 — Structured output

Learn: Pydantic schemas, field validation, enums, and reliable JSON-shaped responses.

Task: Ask the model to classify a short incident report into category, summary, confidence, and evidence_needed fields. Validate the result and handle invalid values.

Deliverable: A Pydantic model and 10 validated sample outputs.

Session 5 — Mini assignment

Learn: Review model calls and structured responses.

Task: Build a command-line incident triage helper that accepts text and returns a validated JSON record. Do not let it send alerts or take external actions.

Deliverable: Week 1 assignment: working CLI, README, and test examples.

Weekly checkpoint: Week 1 assignment: working CLI, README, and test examples.

Week 2 — LangChain building blocks and chains

Goal: use LangChain components to compose predictable model pipelines.

Session 1 — LangChain model interface

Learn: Chat model integrations, messages, invoke, batch, and streaming basics.

Task: Move the Week 1 model call into LangChain. Compare one invoke call and a streamed response.

Deliverable: A small LangChain model adapter with configuration separated from code.

Session 2 — Prompts and runnables

Learn: Prompt templates and composing steps with the current runnable composition style.

Task: Build a prompt → model → parser pipeline for concise case summaries. Pass two different input dictionaries through it.

Deliverable: A reusable chain with input and output clearly defined.

Session 3 — Parsing and validation

Learn: String parsers, structured output, schema validation, and recovery from malformed output.

Task: Return a structured event report with timestamp, event type, observed facts, and uncertainty. Reject missing required fields.

Deliverable: A chain that returns a validated report or a clear error.

Session 4 — Documents and splitting

Learn: Document objects, metadata, loaders, chunk sizes, overlap, and source preservation.

Task: Load a public policy document or your own non-sensitive notes. Split it into chunks and inspect whether headings and page metadata survive.

Deliverable: A script that prints chunk text and useful metadata.

Session 5 — Embeddings and vector search

Learn: Embedding purpose, vector-store interface, similarity search, and retrieval limits.

Task: Embed the chunks from Session 4, run five queries, and inspect the retrieved text. Note one good and one poor result.

Deliverable: Week 2 assignment: local semantic search with source metadata.

Weekly checkpoint: Week 2 assignment: local semantic search with source metadata.

Week 3 — RAG systems and evaluation

Goal: build grounded answers and learn to test retrieval separately from generation.

Session 1 — RAG pipeline

Learn: Indexing, retrieval, context assembly, answer generation, and citations.

Task: Build a basic RAG chain over your course notes. Each answer must name its source document or page when available.

Deliverable: A working document Q&A application.

Session 2 — Improve retrieval

Learn: Chunking experiments, top-k, metadata filters, similarity thresholds, and query rewriting.

Task: Create 10 test questions. Compare two chunk sizes and two retrieval settings. Record which source chunks answer each question.

Deliverable: A retrieval comparison table with chosen settings.

Session 3 — Grounding and abstention

Learn: Separate evidence from inference; handle missing answers; avoid fabricated citations.

Task: Add a rule: when retrieved passages do not support an answer, say the documents do not contain enough information. Test with five unanswerable questions.

Deliverable: An answer format with answer, evidence, sources, and uncertainty.

Session 4 — RAG agent

Learn: When an agent may choose retrieval as a tool, and when a fixed RAG pipeline is enough.

Task: Expose your retriever as a tool. Let the agent use it for questions that need documents; compare results against the fixed chain.

Deliverable: A small RAG agent and a note on when it helps.

Session 5 — Assignment: evidence Q&A

Learn: Combine retrieval, citations, and test cases.

Task: Prepare 15 questions from a public manual or course notes, including answerable, unanswerable, and ambiguous examples. Score source relevance and answer correctness manually.

Deliverable: Week 3 assignment: evaluated RAG app with a 15-question test set.

Weekly checkpoint: Week 3 assignment: evaluated RAG app with a 15-question test set.

Week 4 — Tools and single-agent applications

Goal: understand tool calling and build one agent that uses a small set of safe tools.

Session 1 — Design good tools

Learn: Python callables, tool names, descriptions, argument types, return values, and least privilege.

Task: Create three read-only tools: calculator, document search, and incident-record lookup over sample data. Write precise docstrings.

Deliverable: Three tools with input validation and example tests.

Session 2 — Tool calling loop

Learn: Model requests a tool, application runs it, result returns to the model, and the agent finishes.

Task: Call each tool directly first. Then give the tools to a LangChain agent and inspect which tool it selects for 10 requests.

Deliverable: A tool-choice log showing input, selected tool, and result.

Session 3 — Create a single agent

Learn: Current create_agent pattern, system prompt, tools, invocation, and bounded execution.

Task: Build a case-notes assistant that can search documents and calculate simple values. Limit the tool list and set sensible iteration limits.

Deliverable: A working tool-using agent with five test prompts.

Session 4 — Memory and conversation state

Learn: Thread IDs, short-term conversation context, and the difference between conversation memory and a knowledge base.

Task: Add conversation continuity for follow-up questions. Verify that two different thread IDs do not share history.

Deliverable: A two-thread conversation test.

Session 5 — Assignment: personal research agent

Learn: Combining tools, grounded response, state, and traces.

Task: Make an agent that searches your own study notes, returns cited answers, and asks a clarifying question when the request is unclear.

Deliverable: Week 4 assignment: small research agent and recorded test run.

Weekly checkpoint: Week 4 assignment: small research agent and recorded test run.

Week 5 — LangGraph step by step

Goal: express an application as state, nodes, and transitions, then add persistence and human review.

Session 1 — Graph mental model

Learn: State is shared data; nodes perform work; edges select the next step; START and END define execution.

Task: Draw a graph for a support request: classify → retrieve answer or ask a person → respond. Label each state field.

Deliverable: A graph diagram and state schema.

Session 2 — Build a StateGraph

Learn: Define state, node functions, edges, compile, and invoke.

Task: Implement a graph with intake, classify, retrieve, and answer nodes. Use sample data first; keep model calls in one node.

Deliverable: A runnable graph and printed state after each node.

Session 3 — Branches and loops

Learn: Conditional edges, routing, retry loops, and stopping conditions.

Task: Route supported questions to retrieval, unrelated questions to a polite refusal, and ambiguous questions to clarification. Add a maximum retry count.

Deliverable: A graph with three tested routes and a bounded loop.

Session 4 — Checkpointing and threads

Learn: Checkpoints, thread identifiers, continuing a run, and conversation persistence.

Task: Add a checkpointer. Pause a conversation, restart the process, and confirm whether the same thread can resume with the selected persistence backend.

Deliverable: A persistence test and notes on what is stored.

Session 5 — Interrupts and human approval

Learn: Pause execution for review; resume with a human decision; keep side effects behind approval.

Task: Insert an approval pause before an alert-draft node. Test approve, reject, and edit paths. Do not send a real alert.

Deliverable: Week 5 assignment: graph workflow with resume and human approval.

Weekly checkpoint: Week 5 assignment: graph workflow with resume and human approval.

Week 6 — Different agent and workflow patterns

Goal: know the main patterns and select the simplest one that fits a task.

Session 1 — Workflow versus agent

Learn: A workflow follows developer-defined steps; an agent chooses among available actions. Tradeoffs in cost, speed, predictability, and control.

Task: Implement the same FAQ task once as a fixed workflow and once as a tool-using agent. Compare latency, tool calls, and failures.

Deliverable: A short comparison and a decision rule for each approach.

Session 2 — Router and specialist agents

Learn: Classify intent, route to one specialized chain or agent, and return a consistent result.

Task: Build a router that sends questions to digital forensics, IoT, or general AI notes. Add a fallback for unknown categories.

Deliverable: A router with at least 12 test questions.

Session 3 — Sequential and parallel workflows

Learn: Fixed pipelines, independent parallel tasks, joins, and map-reduce summaries.

Task: Given an incident report, run separate extraction and summarization steps, then combine their structured results. Make clear which steps can safely run in parallel.

Deliverable: A workflow that produces one combined report.

Session 4 — Planner and executor

Learn: Break a task into steps, validate the plan, execute bounded steps, and check completion.

Task: Create a plan for investigating a fictional evidence bundle. Allow only approved read-only tools and cap the plan at five steps.

Deliverable: A plan plus execution record, with any missing evidence stated.

Session 5 — Handoffs and supervisor with subagents

Learn: Transfer control between roles; delegate only when subagents have distinct work; combine results with provenance.

Task: Prototype a supervisor with two narrow specialists: evidence search and timeline summarization. Compare it with a single agent.

Deliverable: Week 6 assignment: pattern comparison and one small multi-agent prototype.

Weekly checkpoint: Week 6 assignment: pattern comparison and one small multi-agent prototype.

Week 7 — Testing, safety, observability, and API serving

Goal: make the application easier to inspect, safer to use, and more reliable under failure.

Session 1 — Evaluation and regression tests

Learn: Golden test cases, expected tool use, answer checks, retrieval checks, and repeatable evaluation.

Task: Create 20 test prompts covering normal, ambiguous, adversarial, and unanswerable inputs. Run them after every prompt or tool change.

Deliverable: A repeatable evaluation script and results file.

Session 2 — Tracing and debugging

Learn: Trace model calls, graph nodes, inputs, outputs, tool calls, errors, latency, and token/cost data.

Task: Enable tracing in a development environment, inspect one successful and one failed run, and remove secrets or sensitive data from logs.

Deliverable: Two trace screenshots or IDs with a short debugging note.

Session 3 — Agent safety and data boundaries

Learn: Prompt injection, untrusted retrieved text, permission boundaries, tool validation, secret handling, and human approval.

Task: Add tests where a retrieved document tells the agent to ignore instructions. Ensure tools are read-only and the agent treats retrieved content as evidence, not instructions.

Deliverable: A safety test set and a list of permitted tool actions.

Session 4 — Reliability and API wrapper

Learn: Timeouts, retries, fallback responses, request validation, logging, and FastAPI integration.

Task: Expose your agent through one FastAPI endpoint. Add request and response schemas, timeout handling, and a request ID.

Deliverable: A locally testable API with sample requests.

Session 5 — Assignment: release candidate

Learn: Combine tests, traces, guardrails, and API.

Task: Run the 20-case evaluation. Fix the most common failure, document known limits, and verify keys are absent from Git and logs.

Deliverable: Week 7 assignment: release candidate with README and evaluation summary.

Weekly checkpoint: Week 7 assignment: release candidate with README and evaluation summary.

Week 8 — Capstone forensic investigation assistant

Goal: build a useful, reviewable assistant that supports investigators without making unsupported claims or taking unapproved actions.

Session 1 — Scope and data model

Learn: Define the user, accepted inputs, evidence sources, outputs, permissions, and non-goals.

Task: Use synthetic or public sample data only. Define schemas for an incident, evidence item, retrieved passage, timeline event, and draft alert.

Deliverable: A one-page design and sample dataset.

Session 2 — Evidence tools and RAG

Learn: Create narrow read-only tools and retrieval over investigation procedures or synthetic case notes.

Task: Implement evidence search, case metadata lookup, timeline retrieval, and procedure search. Include source IDs and timestamps in results.

Deliverable: Tools with tests and source-preserving retrieval.

Session 3 — LangGraph investigation flow

Learn: Represent intake, validation, evidence retrieval, event summary, uncertainty review, and final response as graph steps.

Task: Build a graph that routes incomplete requests to clarification, performs read-only retrieval, and returns a structured evidence summary with citations.

Deliverable: A working graph with successful and incomplete-case runs.

Session 4 — Human-reviewed alert drafting

Learn: Prepare an alert draft only after relevant evidence is found; wait for approval before any sending integration.

Task: Add an interrupt where a reviewer can approve, reject, or edit the alert. Keep actual messaging disabled in the course project.

Deliverable: A tested approve/reject/edit flow and sample alert.

Session 5 — Capstone demonstration

Learn: Evaluate, document, and explain design decisions and limitations.

Task: Run 20–30 test cases including false alarms, crowded scenes, missing timestamps, contradictory evidence, and prompt injection. Present the system architecture and a 5-minute demo.

Deliverable: Final repository, README, graph diagram, evaluation report, and demo.

Weekly checkpoint: Final repository, README, graph diagram, evaluation report, and demo.

Agent patterns reference

“Agent type” is often used informally. The patterns below describe common ways to organize model decisions and application control. They are not all separate frameworks or separate model types.

Pattern	How it works	Use it when
Tool-using agent	The model selects tools, reads results, and continues until it can respond.	The task needs flexible choice among a few well-defined tools.
RAG agent	The agent can decide when to search a document collection before answering.	Questions vary and some require retrieval. Use fixed RAG when every question should always retrieve.
Router	A classifier or model sends each request to one specialist path.	There are clearly different topics, data sources, or permissions.
Workflow	Code controls a known sequence of steps; a model may fill in a step.	The process is predictable and auditability matters.
Planner and executor	A planner proposes bounded steps; an executor carries them out and reports progress.	A task needs multiple steps but must stay within a controlled tool set.
Handoff	One specialist gives control to another based on the conversation or task state.	A request changes role or needs a specialist after initial triage.
Supervisor with subagents	A lead agent delegates focused tasks, then checks and combines their outputs.	Subtasks are genuinely distinct and delegation is worth its added cost and complexity.
Human-in-the-loop	Execution pauses for a person to approve, reject, or edit a proposed action.	An action has real-world impact or needs professional judgment.

Capstone architecture and acceptance checklist

Suggested flow: user request → validate request → retrieve relevant case material → summarize evidence with sources → check uncertainty → draft response or alert → pause for human review. In the surveillance use case, the computer-vision model should produce detections or event candidates; the agent can retrieve context and prepare a reviewable summary. Do not rely on a language agent alone to decide whether people are fighting from raw video.

- Every factual claim in the summary points to an evidence item, timestamp, or source passage.
- The assistant says when evidence is missing, conflicting, or uncertain.
- All evidence tools are read-only for this learning project.
- No message, deletion, account change, or external action occurs without a human approval step.
- Runs have request IDs and traces; secrets and sensitive footage are not placed in logs.

- The test set includes ordinary cases, edge cases, false positives, missing data, and adversarial text.
- The README explains setup, data handling, model choice, graph flow, test results, and limitations.

Suggested grading rubric

Area	Weight	Evidence
Core concepts and clean Python	15%	Small, readable modules; schemas; error handling
LangChain and RAG	20%	Working chains, retrieval, citations, and test examples
LangGraph design	20%	State, nodes, branches, persistence, and approval pause
Agent patterns and tool design	15%	Clear tool descriptions, correct routing, justified pattern choice
Testing, safety, and observability	20%	Evaluation set, trace review, data boundaries, safe defaults
Documentation and demo	10%	Runnable README, architecture diagram, clear demonstration

Official learning resources

Use the official documentation as the source of truth for package installation and APIs. Framework interfaces change; when a code example does not match the installed version, check these pages before copying older tutorials.

- LangChain Learn: https://docs.langchain.com/oss/python/learn
- LangChain agents: https://docs.langchain.com/oss/python/langchain/agents
- LangGraph Quickstart: https://docs.langchain.com/oss/python/langgraph/quickstart
- LangGraph Graph API: https://docs.langchain.com/oss/python/langgraph/graph-api
- LangGraph interrupts: https://docs.langchain.com/oss/python/langgraph/interrupts
- LangChain Academy: https://academy.langchain.com/

Completion checklist

☐ I can explain when a fixed workflow is safer than an autonomous agent.

☐ I can define a tool with validated inputs, a clear description, and a limited permission scope.

☐ I can build and test a grounded RAG application with source references.

☐ I can create a LangGraph with shared state, branches, persistence, and a human approval interrupt.

☐ I can describe the tradeoffs of router, planner/executor, handoff, and supervisor/subagent patterns.

☐ I have a capstone repository, evaluation set, README, and demonstration.